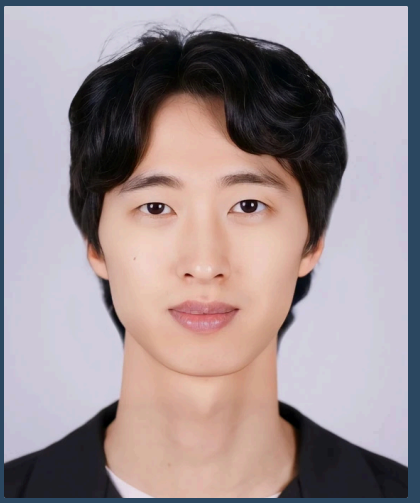
기계공학 전공 / 로봇 분야 신입 지원

정승호 SEUNGHO JUNG



기계공학 지식을 바탕으로,
로봇을 만들고 제어한다.

기계공학을 전공하고 로봇 SW를 배우고 있다. 로봇팔 제작·제어, 비전 인식과 공정 연동에서 계산 기준을 정하고 실물 동작을 확인했다.

3095069@naver.com **GitHub ↗**

프로젝트와 실습에서 사용한 기술

학력·교육 보기 ↗

01 제어·임베디드

C · C++(기초) · STM32

PLC · PWM · I²C · UART

로봇팔 제어와 펌웨어 통합

02 인식·데이터

Python · MATLAB

YOLO · OpenCV · TFLite

진동 신호 분석과 영상 인식

03 설계·개발 도구

SolidWorks · Ansys

Linux · Git · ROS2

전공 실습 · AI 활용 앱 구현

학력과 교육, 자격 사항

기계공학 전공 지식을 바탕으로 로봇 제작과 제어를 학습하고 있다.

학력·교육

2026.02 졸업

순천향대학교 기계공학과

학점 3.65 / 4.5

2026.04.14 – 2026.11.09 교육 중

대한상공회의소 서울기술교육센터

AI융합 로봇 SW개발자 3기

Python·Linux 기반 AI 프로그래밍, ROS/ROS2 통신·제어, Cortex-M·PLC와 산업용 네트워크를 학습하고 있다.

2019.02 졸업

대성고등학교

자격·어학

초경량비행장치조종자

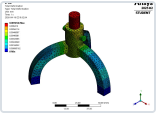
2019.09 · 한국교통안전공단

운전면허 1종 보통

2019.08 · 경찰청

TOEIC 705점

2025.12

설계·해석 실습

이미지 4장 보기 ↗

지그 위치에 맞춰 다시 계산하는 로봇팔 티칭

카메라로 지그의 위치·방향을 측정해 저장한 집기·놓기 동작을 보정한다.

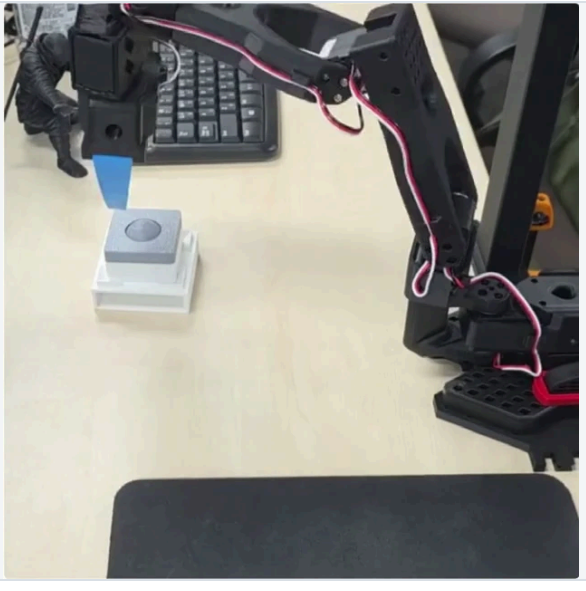


그림 1. 지그 위치 보정 시연 · 편집 영상 · 무음

이동 로봇의 도킹 위치와 각도가 달라지면 저장한 관절값만으로는 집기 위치가 어긋난다. 카메라로 지그를 측정해 조립·하차 동작의 목표를 보정했다.

> **담당 역할**
>
> 6인 최종 프로젝트에서 로봇팔 2·3의 비전 보정·티칭과 Pi 실행 연동을 맡았다. 계산 기준과 GUI 요구사항을 정하고, AI 도구를 활용해 구현·수정한 기능을 화면과 실물에서 확인했다.

Python · OpenCV | FK · IK | SO-101 · Raspberry Pi | ROS2 Action

전체 발표자료 ↗

기간 · 팀	적용 공정	담당 장비	진행 상태
2026.09–11 · 6인	**조립 · 완성품 하차**	**SO-101 팔 2·3 · Pi**	**교육 최종 프로젝트 진행 중**

영상 좌표를 집게 목표와 관절값으로

카메라의 지그 위치 변화가 보정 목표와 3D 자세에 반영되도록 연결했다.

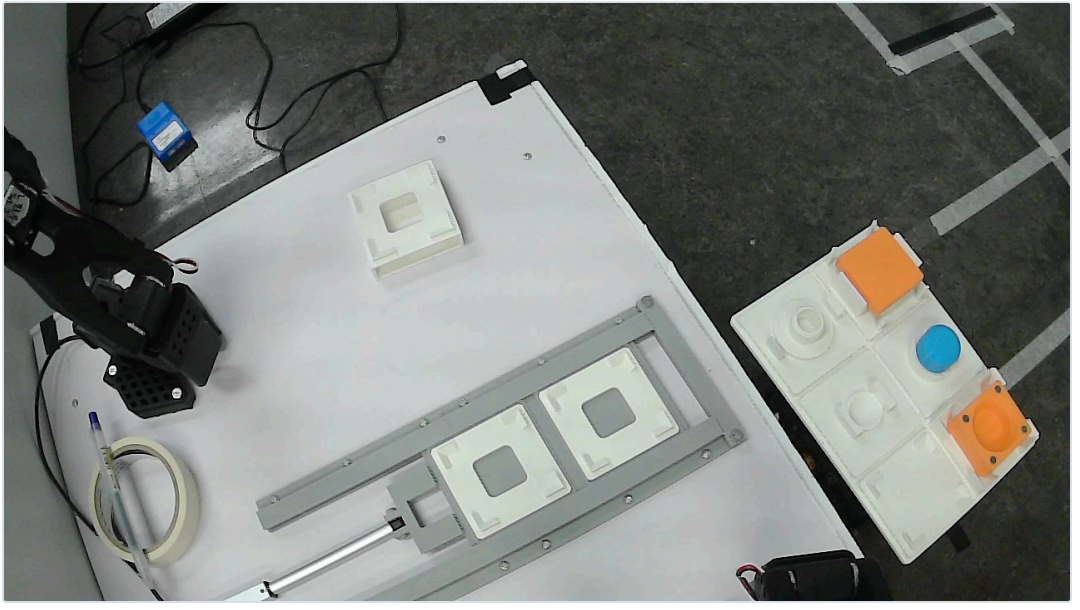


그림 2. 2026.10.14 · 실제 상단 카메라 영상

지그 중심·방향 측정

외곽선·구멍·교차점으로 지그를 찾다. 카메라 왜곡과 지그 높이를 반영해 mm 좌표로 바꾸고, 여러 프레임이 일치할 때 채택한다.

티칭 목표의 이동·회전

순기구학(FK)으로 저장 자세의 집게 끝을 구한다. 지그 변화량만큼 위치·방향을 옮기고, 높이와 집게 벌림 목표는 유지한다.

관절값 계산과 도착 확인

역기구학(IK)으로 관절 범위와 자세 연속성을 고려해 목표를 계산한다. Pi에서 실행하고 모터 실측값으로 도착을 확인한다.

실제 카메라에서 지그 이동을 반영해 새 위치를 채택하고, 보정된 관절 자세가 3D 화면에 갱신되는 것을 확인했다. 저장한 티칭 원본은 유지한다.

티칭·MuJoCo 미리보기·공정 실행 연동

로봇팔별 작업 편집부터 조립·하차 요청, 실행 결과와 오류 기록까지 연결했다.



그림 3. 티칭 스텝과 MuJoCo 자세 미리보기

구분	구현·확인한 범위
MuJoCo	작업대·로봇팔·지그를 3D로 표시하고, 선택 스텝의 보정 자세와 경로를 미리 확인한다.
ROS2·Pi	팀 관제 규격에 맞춰 조립·하차 요청을 Pi 실행기와 연결하고 상태·완료 결과를 반환한다.
검사·기록	부품 안착 검사와 작업 실행을 연결하고, 오류 사진·로그를 로봇팔·날짜·오류별로 조회한다.

AI 활용 요구사항·계산 기준을 정하고 구현·수정에 AI 도구를 활용했다.
실제 카메라와 GUI에서 변경 결과를 확인했다.

모방학습 담당 시연 기록·학습 실행·모델 재생 프로그램과 환경 구성. 시연 데이터 수집·ACT 학습·실물 실행은 팀원이 담당했다.

스마트폰으로 조작하는 이동형 로봇

4륜 이동체와 로봇팔을 결합해 스마트폰으로 주행과 물품 회수 동작을 제어했다.

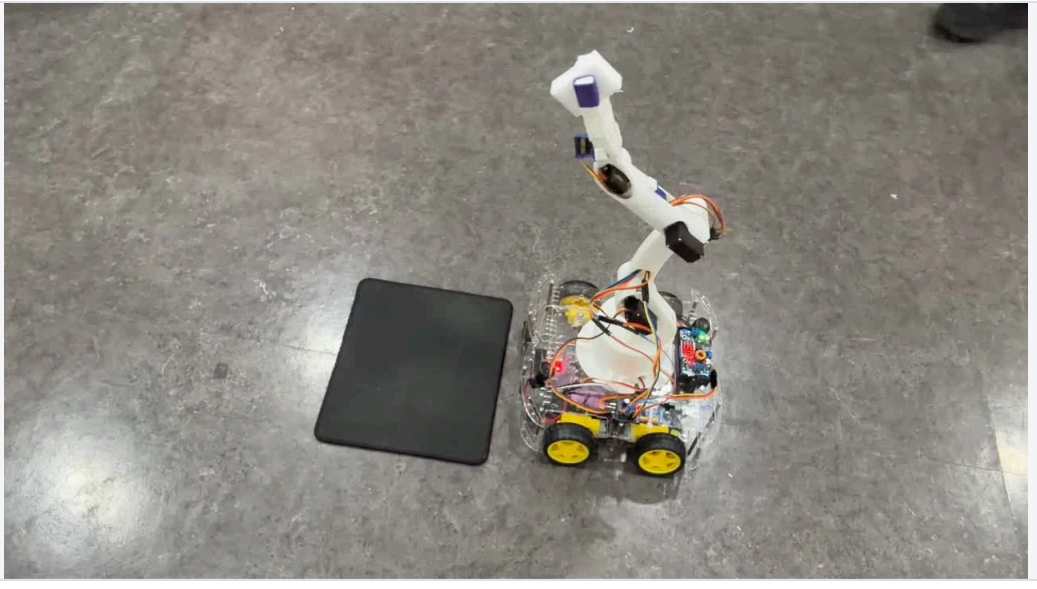


그림 4. 원격 물품 회수·운반 시연 · 무음

이동체에 로봇팔을 결합해 물체를 집고 옮기는 시제품이다. 앱에서 주행과 관절을 조작하며, 미리 저장한 동작을 재생할 수 있다.

> **담당 역할**
>
> 5인 팀의 조장으로 로봇팔 제작·제어와 로봇팔 펌웨어 구축·통합을 맡았다. 팀원들과 통신 규격과 주요 설정을 먼저 정한 뒤, 앱 조정에도 참여했다.

STM32F411 C · FreeRTOS Flutter Bluetooth

전체 발표자료 ↗ **소스 코드 ↗**

수행 시기	팀 구성	구동 구성	구현 범위
2026.07 · 1주	**5인**	**4WD · 서보 6개**	**수동 조작 · 티칭**

로봇팔 제어와 시스템 통신 구조

팀 전체 시스템의 통신 구조이다. 본인은 로봇팔 제어·보정·동작 저장과 펌웨어 구축·통합에 참여했다.

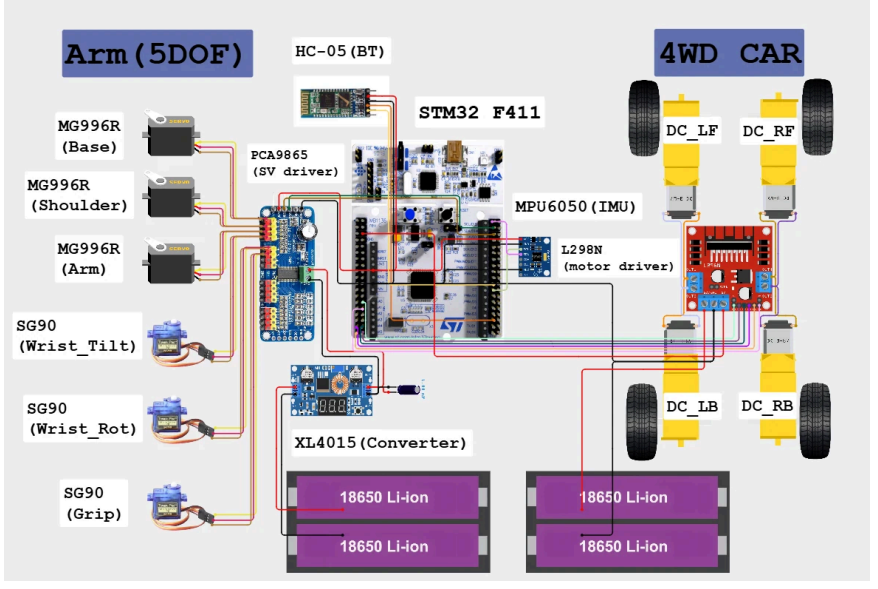


그림 5. 로봇팔·이동체의 전체 배선도

16바이트 명령·응답

주행·로봇팔·티칭·설정을 모드로 구분했다. 앱의 요청과 펌웨어의 응답을 연결해 처리 상태를 확인한다.

서보 제어와 동작 저장

PCA9685로 서보 6개를 제어한다. 티칭·설정은 Flash에 저장하며, 최대 12개 시퀀스 × 30개 웨이포인트를 지원한다.

동작 간 간섭 방지

로봇팔 이동·Flash 작업 중에는 주행을 잠근다. 500ms 동안 유효한 주행 명령이 없으면 이동체를 정지시킨다.

조작 기능과 구동 문제의 개선

로봇팔의 부하 시 떨림과 급격한 관절 움직임을 개선했다. 앱에서 관절 제어·원점 보정·티칭 동작을 확인했다.

01 주행 02 로봇팔 03 원점 보정 04 티칭

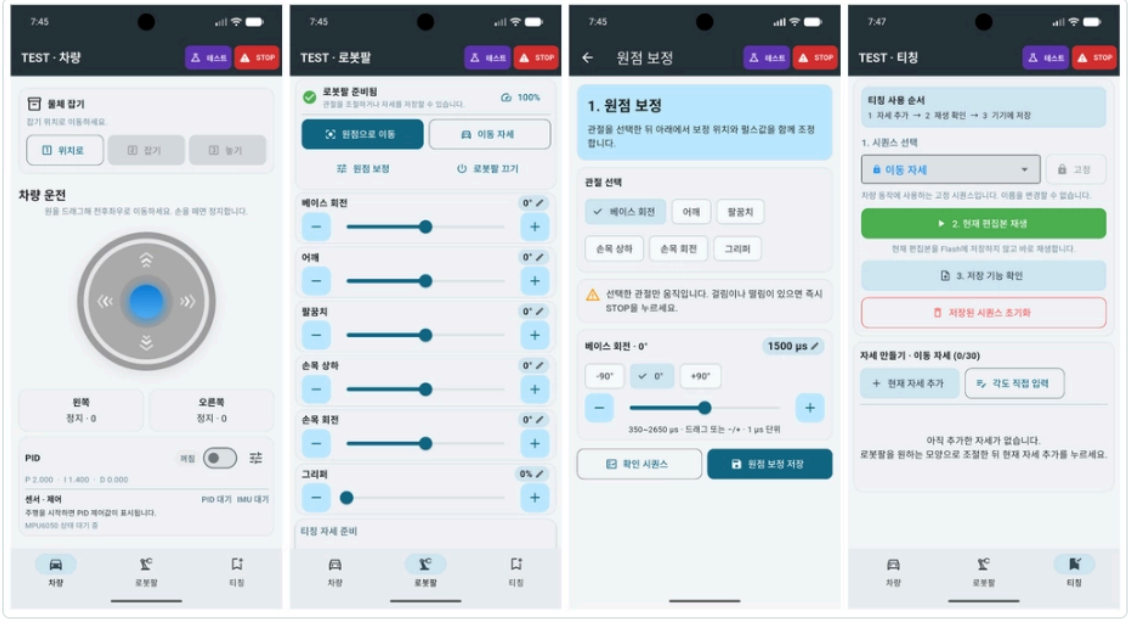


그림 6. 앱 TEST 모드 화면 · 주행 · 로봇팔 · 원점 보정 · 티칭

문제	개선 내용
부하가 걸릴 때의 떨림 전원·서보 개선	전원부에 커패시터를 추가하고, 토크 여유가 있는 서보로 교체해 떨림을 줄였다.
관절의 급격한 움직임 관절 제어 · 공동 구현	20ms 주기의 S-curve를 적용해 관절을 이동시켰다.

객체 탐지와 지면 분할을 이용한 보행 보조

라즈베리파이 5에서 객체 탐지와 지면 분할 모델을 실행한 팀 프로젝트이다.



그림 7. 녹화 영상의 객체·지면 추론 시연 · 무음

영상에서 사람·차량·장애물을 탐지하고 인도와 차도를 구분했다. 보행 경로에 가까운 대상을 판단해 음성으로 알리도록 했다.

> **담당 역할**
>
> 학습 데이터의 라벨 통합, 모델 학습·변환, 실행 환경 구성을 맡았다. 보행·차량 ROI와 알림 판단 로직을 구현하고 녹화 영상으로 동작을 확인했다.

Python · OpenCV | YOLO26n | TFLite INT8

전체 발표자료 ↗ **담당 구현 코드 ↗**

수행 시기	팀 구성	객체 라벨 통합	지면 라벨 통합
2026.05 · 11일	**4인**	**29종 → 10종**	**23종 → 7종**

라즈베리파이 실행 환경에 맞춘 조정

입력 해상도와 모델 형식을 조정하고, 보행 경로와 차량 접근을 판단할 영역을 정했다.

입력 해상도 축소와 모델 변환

입력 영상의 해상도를 1920×1080에서 512×288로 줄이고, 모델을 INT8 TFLite로 변환했다.

보행·차량 ROI 분리

보행 ROI는 발 앞쪽에 해당하는 화면 하단 중앙에 배치했다. 옆에서 접근하는 차량은 더 넓은 ROI로 판단했다.

녹화 영상 기반 동작 시험

녹화 영상을 재생하며 모델의 추론 동작을 확인했다. 객체는 4fps, 지면은 3fps로 추론 빈도를 제한했다.

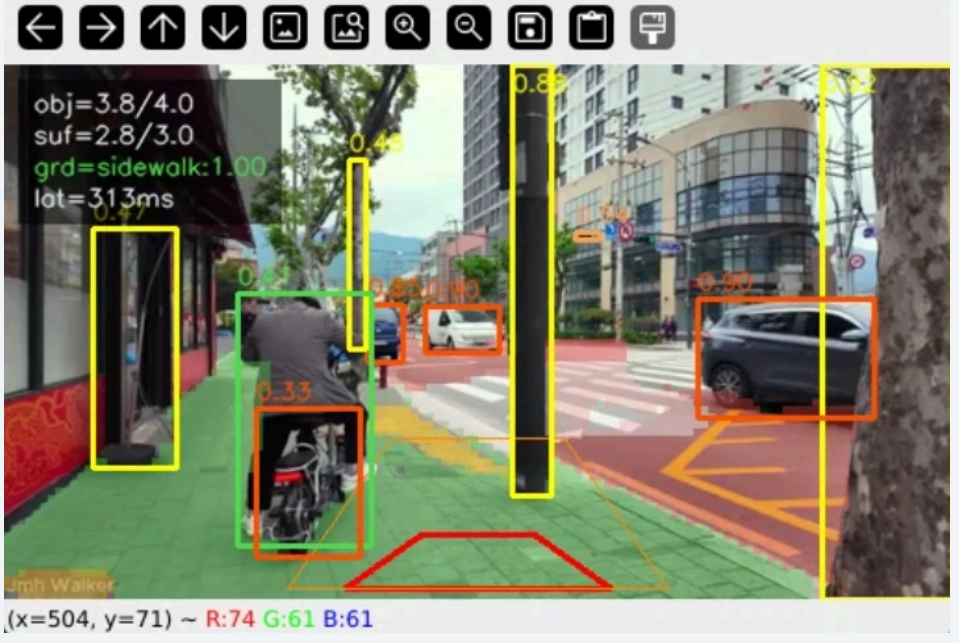


그림 8. 빨간색: 보행 경로 · 주황색: 차량 접근 영역

소재 공급부터 창고 적재까지의 공정 제어

MPS 트레이너에서 소재 분류와 1–4층 창고 적재·비움 동작을 구현했다.

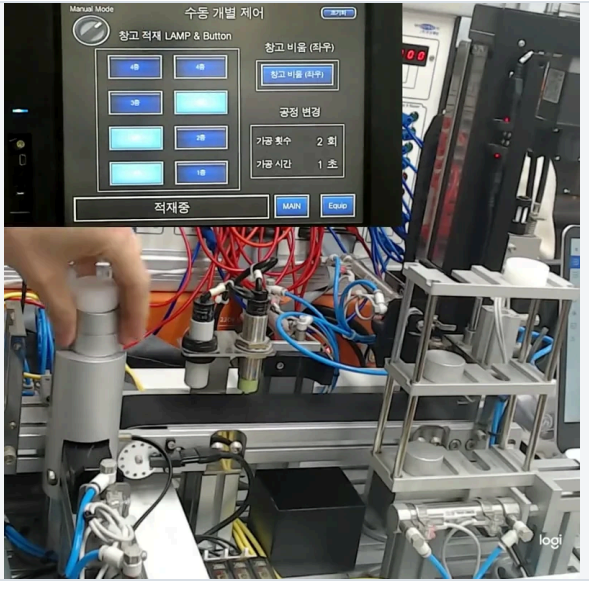


그림 9. PLC 창고 적재 공정 시연 · 무음

컨베이어·실린더·서보모터로 공급·가공·분류·저장 공정을 연결했다. 여러 장치를 병렬로 운전하고, 수동 모드에서는 개별 제어할 수 있도록 했다.

> **담당 역할**
>
> 병렬 처리 로직과 HMI 화면을 작성했다. 작성한 로직을 팀원의 수동 제어 코드와 통합하는 과정에서 메모리 상태가 남아 발생하는 오동작을 수정했다.

Mitsubishi Q PLC · GX Works2 · GT Designer3

전체 발표자료 ↗

수행 시기	팀 구성	래더 프로그램	HMI 작화
2026.06 · 3일	**2인**	**GX Works2**	**GT Designer3**

병렬 시퀀스의 상태 관리와 코드 통합

저장 가능 여부를 판단하는 메모리와 실제 적재 정보를 분리해 시퀀스 간 간섭을 줄였다.

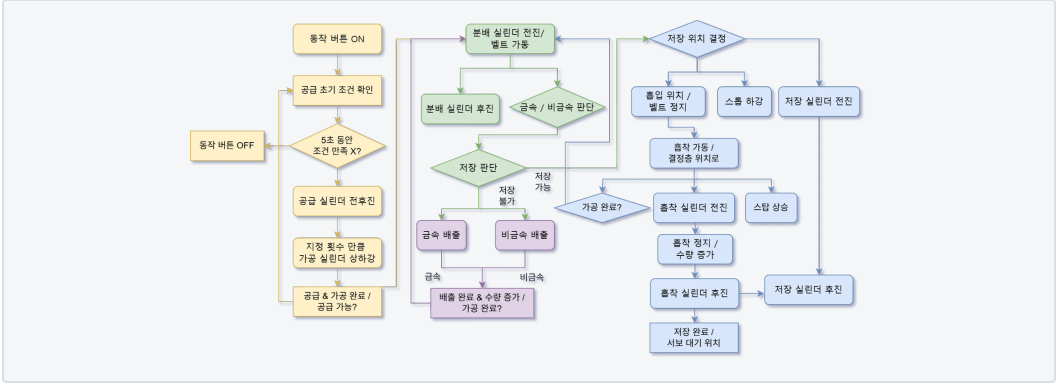


그림 10. 저장 판단과 적재 동작의 병렬 시퀀스

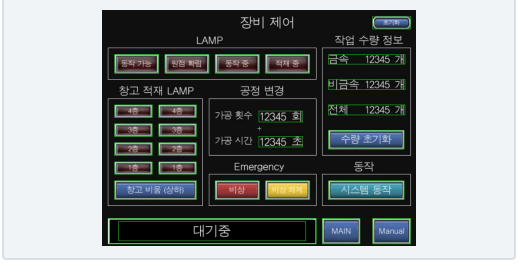


그림 11. 자동 운전·적재 상태



그림 12. 위치·속도 티칭

문제	메모리 저장·초기화 시점이 맞지 않아 **이전 동작의 상태가 다음 시퀀스의 판단에 영향**을 주었다.
개선	**판단용·적재용 메모리를 분리**하고, 동작 조건과 완료 시점에 맞춰 저장·초기화하도록 수정했다.
통합 시험	조건별 동작을 확인하고 **팀원 코드와 겹치는 메모리 주소를 정리**했다.
배운 점	코드 통합 전에 **사용 주소를 합의하는 것이 중요**하다는 점을 배웠다.

피에조 센서를 이용한 노크 위치 탐지

센서 간 시간 차와 시간·주파수 영역 특징을 입력으로 사용해 기계학습 모델을 비교했다.



그림 13. 노크 위치 실시간 탐지 시연 · 무음

아크릴판을 두드릴 때 발생하는 진동을 8개의 피에조 센서로 수집했다. 수집한 신호를 분석해 노크 위치를 9개 구역으로 구분했다.

> **담당 역할**
>
> 노크 위치 탐지를 제안하고 역할을 나눴다. MATLAB으로 신호 전처리·특징 추출, 모델 비교와 실시간 탐지 프로그램을 구현했다.

MATLAB | FFT · 신호 처리 | K-NN · SVM · ANN

전체 발표자료 ↗

논문 제출	팀 구성	수집 데이터	기록 조건
2023.05	**6인**	**9구역 × 140회**	**2,500Hz · 0.4초**

입력 특징별 K-NN 검증 결과

같은 아크릴판에서 수집한 데이터를 학습 70%, 검증 30%로 무작위 분할해 비교했다.

신호 수집부터 실시간 탐지까지

61Hz 하이패스 필터와 데이터 평탄화를 적용한 뒤, 특징을 추출·정규화해 모델을 학습했다.

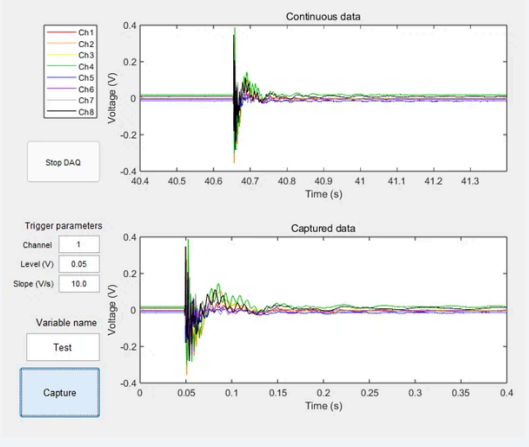


그림 14. 연속 신호와 트리거 기록 화면

학습한 K-NN을 위치 탐지 프로그램에 적용해 노크 위치를 표시하고 데이터를 기록했다.

K-NN 검증 정확도

입력 특징	정확도
센서 간 피크 시간 차	90.741%
시간 영역 특징	96.296%
주파수 영역 특징	97.884%

이동 경로와 사진을 연결한 여행 기록

여행 사진을 이동 경로와 함께 돌아보고 싶어 만든 웹앱이다.

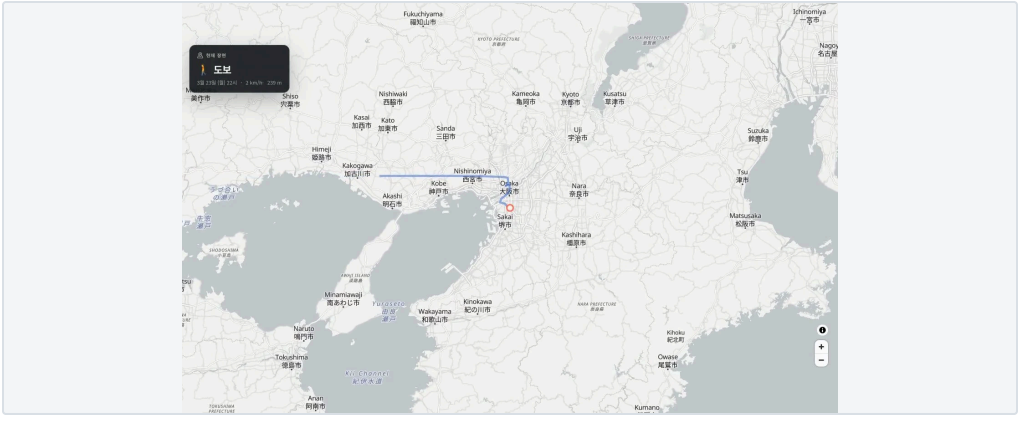


그림 15. 경로 보기 · 이동 기록을 따라가는 지도

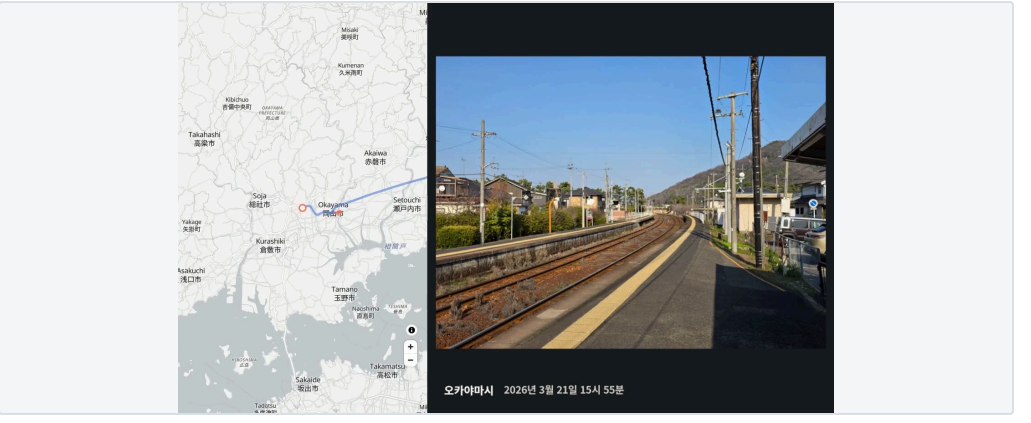


그림 16. 사진 여정 · 촬영 순서로 이어지는 장면

구현 기능

Google Timeline의 이동 기록에 사진·영상을 연결했다. 경로 보기와 사진 여정 모드를 구분하고 PC·모바일 화면에 맞게 배치했다.

React · TypeScript | MapLibre | Node.js

담당 역할

화면·기능을 기획하고 AI 도구를 활용해 구현했다. 실제 여행 기록으로 사용하며 오류를 재현하고, 기대하는 동작과 수정 결과를 비교했다.

소스 코드 ↗

사용 중 발견한 문제와 개선 과정

기본 기능을 만든 뒤에는 오류를 수정하고 사용성을 다듬는 데 더 많은 시간을 들였다.

01
이동 기록·사진
Timeline JSON과
사진·영상 정보 입력

›

02
경로·장면 구성
촬영 시간·위치 연결
여행 선택과 경로 계획

›

03
경로·사진 재생
모드별 재생 위치 유지
화면 크기에 맞춘 배치

앱의 주요 기능과 처리 흐름이다.

세부 동작 수정

모드 전환 후에도 보던 위치를 유지하도록 동작 기준을 정했다. AI 도구를 활용해 수정하고, 실제 여행 기록으로 재생·전환 동작을 확인했다.

불확실한 이동 기록 처리

이동 기록이 끊기거나 GPS와 위치 정보가 크게 다를 때는 이동 수단과 상세 장소를 단정하지 않도록 했다.

개인 데이터 처리

사진과 Timeline 원본은 외부에 올리지 않고 로컬에서 처리한다.